

osmAG: Hierarchical Semantic Topometric Area Graph Maps in the OSM Format for Mobile Robotics

Delin Feng, Chengqian Li, Yongqi Zhang, Chen Yu, and Sören Schwertfeger

Abstract—Maps are essential to mobile robotics tasks like localization and planning. We propose the open street map (osm) XML based Area Graph file format to store hierarchical, topometric semantic multi-floor maps of indoor and outdoor environments, since currently no such format is popular within the robotics community. Building on-top of osm we leverage the available open source editing tools and libraries of osm, while adding the needed mobile robotics aspect with building-level obstacle representation yet very compact, topometric data that facilitates planning algorithms. Through the use of common osm keys as well as custom ones we leverage the power of semantic annotation to enable various applications. For example, we support planning based on robot capabilities, to take the locomotion mode and attributes in conjunction with the environment information into account. The provided C++ library is integrated into ROS. We evaluate the performance of osmAG using real data in a global path planning application on a very big osmAG map, demonstrating its convenience and effectiveness for mobile robots.

I. INTRODUCTION

Autonomous mobile robots have found extensive applications in various fields such as agriculture, industry, and commerce. These robots rely on known maps for autonomous localization, mission-planning and navigation to accomplish tasks. A well-defined map representation serves as a prerequisite for mobile robots to execute their tasks effectively. Given the substantial differences in scale and structural characteristics between indoor and outdoor environments, map representations for mobile robots vary significantly. For outdoor scenarios, large-scale vector maps like OpenStreetMap are common. Point cloud maps are often employed in smaller outdoor scenarios [1]. In indoor settings, 2D or 3D grid maps and point cloud maps are the most widely used map representations. Consequently, most robots are designed for either indoor or outdoor environments exclusively.

With the further advancement of sensor technology, maps are becoming more accurate and detailed. Simultaneously, improvements in simultaneous localization and mapping (SLAM) algorithms have led to increased precision and scalability in map construction. However, memory and computation constraints limit the use of point cloud and 3D grid map based approaches for large areas for localization. They are also typically too detailed for most semantic information that is to be encoded, like room numbers or terrain types. Finally, 3D mesh and point cloud based maps are quite difficult to



Fig. 1. Two buildings with two stories each rendered from osmAG and with a planned path utilizing an elevator from one building to the other.

employ for path planning and navigation. Therefore, for most of these tasks, 2D grid maps are still widely used in robotics. They are suitable up to a certain size in terms of storage and computation, but become computationally more demanding as map sizes grow. They do not readily provide semantic information and, most crucially, are inherently 2D, so fail to support multi-level scenarios like buildings with more than one floor.

Graph structures encoding the topology of an environment are a solution to these problems [2]. These graphs represent places (vertices) and how they are connected (edges). Topometric maps additionally encode metric information about the position of these elements. These graphs can also be easily organized in a hierarchical manner, grouping several places or connections into a parent structure. Adding semantic information to the graph elements further enhances the utility of the map. In our paper [3] we go further into details about hierarchical topometric representation of 3D robotic maps.

Delin Feng, Chengqian Li, Yongqi Zhang, Chen Yu, and Sören Schwertfeger are with the Key Laboratory of Intelligent Perception and Human-Machine Collaboration (ShanghaiTech University), Ministry of Education, Shanghai, China {fengdl, lichq, zhangyq, yuchen, soerensch}@shanghaitech.edu.cn

In our previous work [4] and [5] we developed the so-called Area Graph, a topometric representation generated from 2D grid maps. That work is based on a so-called TopologyGraph [6], which is a pruned and simplified version of the Voronoi Diagram. The Area Graph has as graph nodes areas and as edges passages between the areas: passages mark where it is possible for a robot to traverse from one area to another area that is touching the first area.

In this paper we present an on-file format to store said Area Graph. Additionally, we define how a hierarchy of areas and passages can be established and how semantic information can be added to it. That on-file format is based on and fully compatible with the open street map XML format - we just define certain use patterns and special osm tags that encode the hierarchical, semantic Area Graph structure in the osm XML tag “way”, that is used for both areas and passages. We also provide the osmAG C++ library to load osmAG files into memory and utilize them for visualization and planning in the Robot Operating System (ROS). Fig. 1 shows a multi-story osmAG map and a planned path.

Adopting the open street map XML standard has advantages in its widespread use and support from mature software, facilitating map editing, visualization, updates, and corrections. Apart from autonomous cars and a few other notable exceptions [7], open street map has not been widely adopted in the robotics community, even though some indoor approaches exist [8], [9].

The AreaGraph encodes the areas of rooms based on the architectural walls. We can create the AreaGraph from complete 3D point clouds [3] or from 2D grid maps [4], [5]. We obtain furniture-free 2D grid maps by either employing according to Simultaneous Localization and Mapping technologies [10], [11] or by rendering CAD building data into 2D grid maps. The memory consumption of our abstract osmAG is orders of magnitude smaller than 2D or 3D grid maps and point clouds, and planning on them is easy and fast, as our experiment will show. In [12] we already utilize osmAG for localization and tracking of a mobile robot with a 3D LiDAR, see Fig. 2.

The key contributions of this work are:

- Definition of an on-disk map representation osmAG to store hierarchical, semantic, topometric map data in the open street map format with additional robotics-centric Area Graph data.
- Providing a C++ library osmAGlib to load osmAG into memory and perform tasks like visualization in ROS and path planning.
- Providing a big osmAG map and performing various experiments with it ¹.

¹<https://github.com/STAR-Center/osmAG>

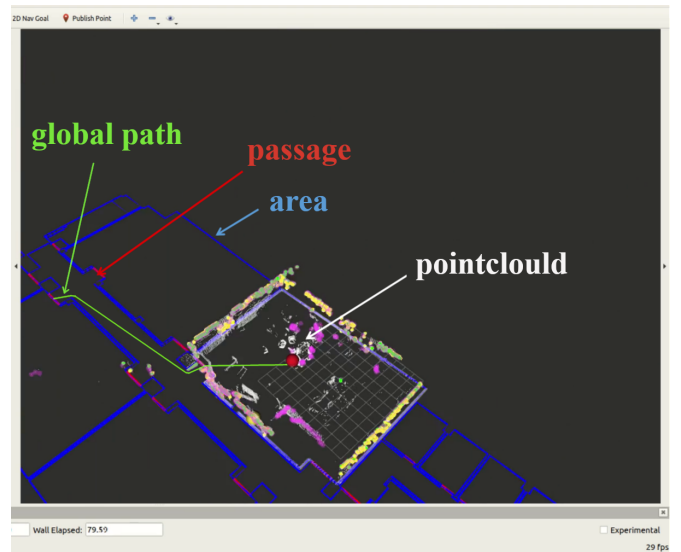


Fig. 2. Application of osmAG for robot localization with a 3D LiDAR pointcloud in the Robot Operating System ROS [12]

II. RELATED WORK

A. Hierarchical Topological Maps

In recent years, there has been great interest in the integration of hierarchical, semantic, and topological mapping. Specifically, learning-based semantic segmentation has made significant progress, enabling high-precision semantic segmentation of 3D point clouds and images. Current research on semantic maps mainly focuses on object-level [13], [14] or dense maps, which include volumetric models [15], point clouds [16], [17], and 3D grids (voxels) [18]. These methods do not involve the estimation of higher-level semantics, such as rooms, and often return dense models that are not suitable for direct navigation. Segmentation alone also does not provide topological connections of neighboring areas.

The second research line focuses on constructing hierarchical topological map models. Hierarchical maps have been widely used since the inception of robotics [2], [19]. Early works focus on 2D maps, exploring the use of multi-layer maps to address the apparent discrepancy between metric and topological representations [20], [21].

More recently, 3D Scene Graphs have been introduced as expressive layered models for 3D environments. A 3D Scene Graph is a hierarchical graph where nodes represent spatial concepts at multiple abstraction levels and edges represent relationships between concepts. Armeni et al. [22] pioneer the use of 3D Scene Graphs in computer vision, modeling the environment as a graph that includes low-level geometry (i.e., metric-semantic grids), objects, rooms, and camera poses, and presented the first algorithm to parse metric-semantic 3D grids into 3D Scene Graphs.

Rosinol et al. [23] propose a novel 3D Scene Graph model that directly builds from sensor data, including subgraphs for places (useful for robot navigation), modeling objects, rooms, and buildings, and capturing moving entities in the environment.

The work most similar to ours is a semantic SLAM method Hydra [24]. Hydra constructs a real-time multi-level 3D Scene Graph using onboard sensors. It transforms a local Euclidean Signed Distance Function (ESDF) into a metric-semantic 3D grid and a Voronoi diagram. It extracts a topological map of places and uses a method inspired by community detection for room segmentation. Our proposed osmAG has a similar hierarchical structure, but we construct the map offline. Unlike Hydra’s oversegmented room segmentation, we accurately segment areas. While Hydra focuses on fine-grained indoor object mapping, we target large indoor and outdoor environments, including multi-floor scenes with adaptable layering.

Another similar research direction is parsing the layout of buildings from 2D or 3D data. A considerable amount of work has been dedicated to parsing 2D maps [25], including hypothesis-based methods [26] and learning-based approaches [27]. One common method for constructing topological maps from 2D occupancy grid maps is to compute Voronoi diagrams, which facilitate the extraction of structures such as points and regions as topological nodes. Depending on the extracted structures, related research can be roughly divided into three types: selecting key points of the Voronoi diagram, such as region boundaries (e.g., doors), to divide the environment into disjoint regions [28], [29]; using the vertices of the Voronoi diagram as nodes in the topological map [6], [30]; performing region segmentation based on the Voronoi diagram, treating regions as nodes, as demonstrated in the work by Friedman et al. [31] using Conditional Random Fields for labeling.

In addition to Voronoi-based methods, Bormann et al. [25] introduced several approaches to parse 2D maps into region nodes such as rooms, including morphology-based segmentation, distance transform-based methods, and feature-based segmentation. Hou et al. [4] proposed a method based on Voronoi diagrams to construct a topological graph with regions as nodes and corridors as edges, using a non-heuristic region growing approach to identify meaningful areas such as rooms. This method is used as the basis for the automatic generation of the topological graph in this paper. Recent work has focused on 3D data. Liu et al. [27] and Stekovic et al. [32] projected 3D point clouds onto 2D maps, but this approach is not suitable for multi-story buildings. Zheng et al. [33] detected rooms by performing region growing on a 3D metric-semantic model.

III. OSMAG

In this section, we will provide a detailed exposition of the structure and composition of the osmAG file format. Our format utilizes the open street map XML tag “way” for areas and passages of the Area Graph, standardizes the use of some common attributes for the XML way elements and introduces a couple of osm tags. osmAG maps can be embedded into normal osm maps - since osmAG is fully compatible with osm.

```

1 <?xml version='1.0' encoding='UTF-8'?>
2 <osm version='0.6' generator='JOSM'>
3   <node id='-286717' action='modify' visible='true' lat='
4     31.17947960453' lon='121.59139728492' root='true' alt='
5     0.00' x='0' y='0' z='0' qx='0' qy='0' qz='0'>
6     <tag k='name' v='root' />
7   </node>
8   <node id='-286718' action='modify' visible='true' lat='
9     31.17950140523' lon='121.59140003821' />
10   <!-- many more nodes -->
11   <way id='-177794' action='modify' visible='true'>
12     <nd ref='-286718' />
13     <!-- many more nodes -->
14     <tag k='height' v='3.2' />
15     <tag k='indoor' v='room' />
16     <tag k='level' v='2' />
17     <tag k='name' v='E1-ST-09' />
18     <tag k='osmAG:id' v='-177794' />
19     <tag k='osmAG:areaType' v='room' />
20     <tag k='osmAG:parent' v='-177816' />
21     <tag k='osmAG:type' v='area' />
22   </way>
23   <way id='-179103' action='modify' visible='true'>
24     <nd ref='-287062' />
25     <nd ref='-287061' />
26     <tag k='level' v='2' />
27     <tag k='name' v='1' />
28     <tag k='osmAG:from' v='-177794' />
29     <tag k='osmAG:to' v='-177814' />
30     <tag k='osmAG:type' v='passage' />
31   </way>
32 </osm>

```

Fig. 3. Non-functional demo of the osm xml of osmAG

A. Definition

First we define some basic osmAG elements, followed by their on-disk osm XML storage format (see Fig. 3). **In the beginning we just consider a non-hierarchical map - the more complex definition for the hierarchy follows afterwards.**

Area: The node of an AreaGraph that encodes an area as a closed polygon, where the points are stored in an ordered list of nodes. Often, especially indoors, areas represent rooms or corridors. The construction algorithm has a parameter that decides how to segment the space (minimum free distance). But areas can also represent whole buildings, certain types of streets or terrains or a whole campus.

Inner area: A free area surrounded by a boundary, e.g. a room, such as the blue outline in Fig. 2. This information is useful for example for localization, where you want to know if you should match against this wall from the inside (inner area) or the outside.

Structure area: The space containing a closed boundary, which is the outer contour of the area and usually is a parent of multiple sub-areas, for example the outer walls of a building.

Passage: The edge of the Area Graph, connecting two areas topologically. The two areas have to share at least two points (nodes). Those shared points form the passage, which is represented as a set of points forming a line.

The on-file format of osmAG follows the open street map (osm) standard with some extensions:

XML tag: node: A geographic location as latitude and longitude coordinates with an id.

Root node: A node tag that contains the pose of the map origin. It converts the WGS84 geodetic coordinate system to Cartesian coordinates as a reference point and facilitates traditional robot indoor navigation.

XML tag: way: An ordered lists of nodes - the osm method to encode both streets and areas.

osm way key: osmAG:id: While osm is unaware of our tags, we cannot utilize the osm ids for ways, as they may change when uploading the data to an osm server. Therefore, we create our own ids for them.

osm way key: osmAG:type: If present that osm way is part of the Area Graph. The value encodes whether this way is an area or a passage.

osm way key: osmAG:areatype: Inner or structure, see above.

osm way keys: osmAG:from and osmAG:to: While the Area Graph is undirected, for the on-file format we save the two osmAG:ids of the two areas connected to a passage in the values of the osmAG:from and osmAG:to keys in arbitrary order.

osm way key: osmAG:parent: This is the key that encodes the hierarchical structure. Its value is the osmAG:id of this area's parent. The areas of all children have to be fully contained within their parent. Areas can never overlap, unless they are on different elevations or are parent and (grand-)child.

This spans a hierarchical tree of areas and their children. One osmAG file can contain multiple such trees, together with traditional osm vector data.

The hierarchical structure is mainly used for two scenarios: First, we may want to join multiple neighboring areas into one bigger area, e.g. to facilitate faster planning and encode bigger areas and semantic tags for them. For example, we can, in the outdoor case, join areas for a parking place and neighboring areas for streets together to a bigger area. Indoors we can join multiple rooms to form a “floor” of a building.

In the second case, the hierarchical structure is used to stack several 2D areas on top of each other, e.g. multiple floors of a building. The height of the floors is encoded, following the osm standard, as height above the ground floor in the osm key “height”.

The different levels of a building are connected via passages. E.g. an elevator will have areas on every floor that are connected to each other vertically by special elevator passages and that are also connected in the 2D plane to one (or more) neighboring areas with passage(s) that have the semantic information “elevatordoor”. Stairs follow the same principle. More details can be found on GitHub.

B. Visualization and Editing

Since osmAG follows the basic data format of open street map, users can perform visualization and editing operations in any software or application that supports OpenStreetMap.

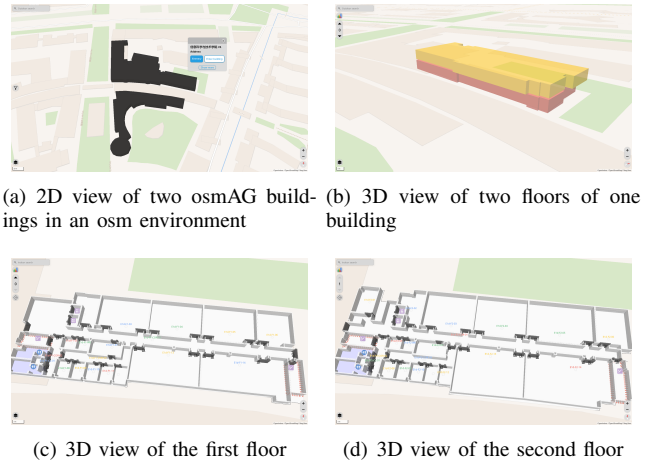


Fig. 4. 3D Rendering of an osmAG map

The three-dimensional rendering of osmAG data was performed in the OpenIndoor application². Fig. 4 (a) illustrates the shape and geographic location of the indoor building. Fig. 4 (b) showcases some floors within one of the buildings, highlighting variations in the structural layout between floors. Fig. 4 (c) and Fig. 4 (d) depict the rendered floor plans of different levels, with black sections representing passages. Walls with a fixed thickness show the area polygons, and semantic labels such as “stairs”, “restrooms”, and “elevators” are annotated according to the icons used in OpenStreetMap.

The two-dimensional visualization and manual editing of osmAG are achieved through the JOSM (Java OpenStreetMap Editor) application. By using JOSM, users are able to easily view, edit, and update the area graph, as shown in Fig. 5.

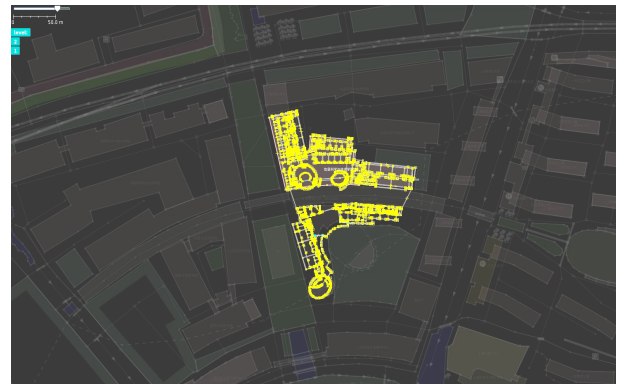


Fig. 5. 2D visualization of osmAG (highlighted) of two buildings in JOSM, together with classic osm data

C. Automated Generation

Although existing editors like JOSM can be used to edit area map, in this case, the elements, information, and relationships of the topological map must be manually added. It is impractical when the scene is vast. We focus on the basic

²<https://wiki.openstreetmap.org/wiki/OpenIndoor>

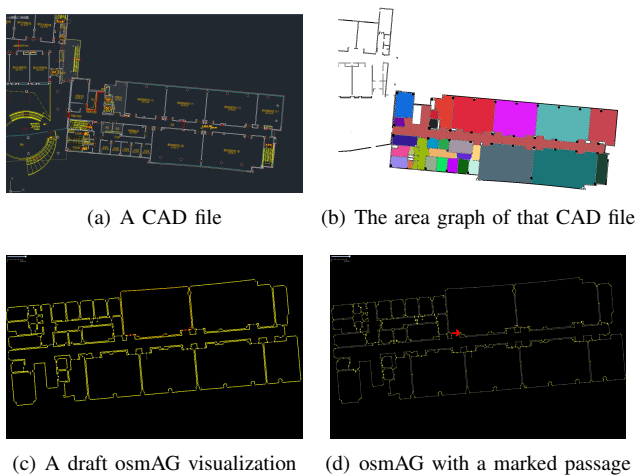


Fig. 6. Generating osmAG from CAD data

functionality of offline mapping for osm. This functionality is implemented by a C++ library based on osmAG.

Offline mapping involves segmenting, extracting, and merging from existing maps to create a new map. These maps can be CAD files of buildings, 3D point clouds, or maps generated by robots. We implement the automatic conversion from 2D grid maps and 3D point cloud maps into osmAG, as well as the simple fusion of multiple topological maps.

Generating from a 2D grid map. Our previous work [4], [5] generates non-hierarchical areas graphs from 2D grid maps. We modified that work to output osmAG files that can then be further edited.

Generating from a CAD file. The process is illustrated in Fig. 6 (a), taking a CAD file input as an example. Generally, CAD files for building layouts contain multiple layers and semantic information. Currently, CAD files are processed manually by removing layers. Fig. 6 (b) shows the generated area graph, where it can be observed that a corridor in the bottom left of the current image is divided into several areas. By adjusting the parameter *alpha* shape in the algorithm, issues of over-segmentation and under-segmentation can be alleviated. Fig. 6 (c) demonstrates the effect of directly using the Area Graph method to generate the topological map's area nodes. Fig. 6 (d) illustrates how this method correctly generates inner areas and passages for the osmAG (the red arrow in the image represents a passage).

Generating from a 3D pointcloud map. We follow our previous work [3] which generates hierarchical 3D topometric maps from 3D point clouds of the whole building, to extract 2D Area Graphs from a 3D pointcloud map.

These extraction methods are suitable for generating topological representations in indoor environments. For further processing, our osmAGlib C++ library efficiently merges multiple maps. This is achieved through a process that involves assessing the distances between nodes and selectively consolidating certain points to merge osmAG maps and integrate existing outdoor OpenStreetMaps into the current map.

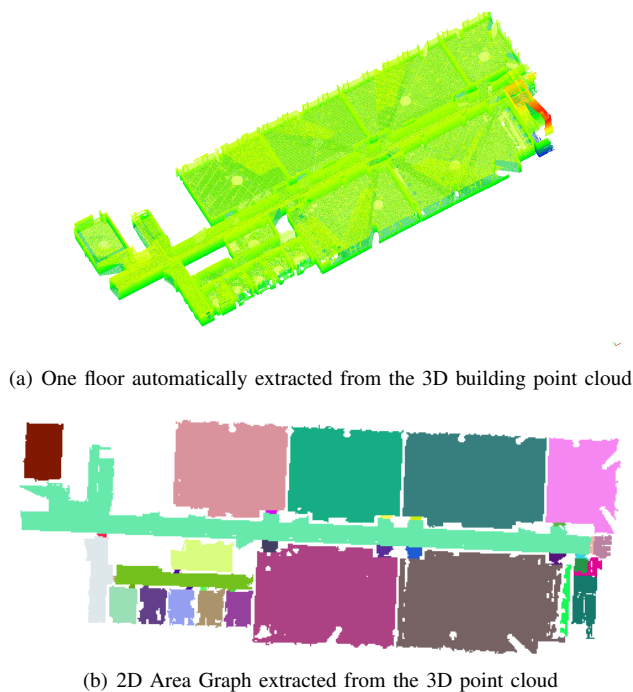


Fig. 7. The automatic generation of osmAG from 3D maps[3]

IV. PATH PLANNING WITH OSMAG

Navigation is an essential part of mobile robotics and utilizing osmAG for path planning is a crucial part in developing a navigation stack for osmAG. We have already demonstrated the effectiveness of path planning in the area graph [34]. Here, we describe our approach to planning with the hierarchical area graph data structure.

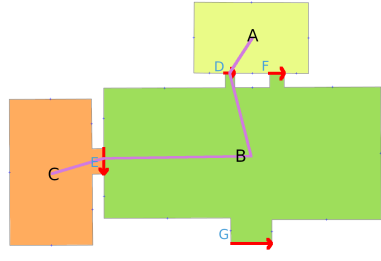
The goal of path planning in osmAG is to provide the best global path between two coordinates. The paths generated may be close to walls and it is the job of local planning to keep a safe distance during navigation.

We cannot use the graph structure of the area graph to directly plan, because the cost is in traversing the vertices, while the cost of traversing a passage is typically zero. The cost of traversing the area depends on which passage you enter and through which passage you exit. A simple solution would assign the cost by summing up the distance from the first passage to the center with the distance from the center to the second passage. But that may overestimate the cost and lead to sub-optimal paths as shown in Fig. 8 (a).

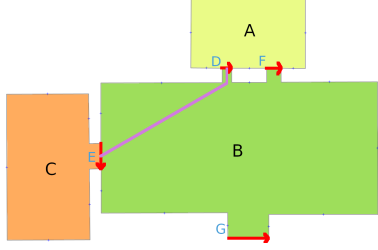
We therefore opt for generating a 2D grid map just from that area and utilizing A* on that grid map to calculate the true cost for that traversal (Fig. 8 (b)). Consequently, for planning we build a new graph where the passages are the vertices and where we add edges to all possible other passages of that area.

We pre-compute this passage graph on the leaves of the whole Area graph. Planning a path between two coordinates then follows these simple steps:

- 1) Find the leaf areas of the start and goal points.
- 2) (Special case: if both are in the same area, render that area and use A* on that grid map)



(a) Planning through the center of an area



(b) Planning true cost with rendered 2D grid map

Fig. 8. Comparison of cost calculation options during path planning.

- 3) Temporarily add edges from the start (goal) point to all passages of that area by using A* on the rendered 2D grid map of that area.
- 4) In that graph use A* to search for a path from the start to the goal node.
- 5) Return the path in terms of a list of passages and areas and/or in terms of a set of points (from the A* of the pre-computed 2D grids).

A. Speedup using Hierarchy

Path planning can be further sped up by utilizing bigger, higher-level areas. This is done by pre-computing the cost of all combinations of start and end passages of each non-leaf area. This pre-computation is very fast, since it utilizes the already computed costs of the child areas. During planning, whenever we encounter a passage we select the highest level area it connects to that is not containing the goal point - unless it is the goal leaf area.

B. Planning using Semantic Information

One great feature of planning with the osmAG map is, that it can take robot-specific capabilities into account. For that semantic cues from the passages and areas are taken into account. For example, a wheeled robot will not plan through an area with the semantic information “stairs”, while a legged robot might. For that we plan to callback a cost-calculation function for every area and passage that is encountered, so robot specific code may determine the cost. The legged robot may return the cost in terms of the estimation of time needed to traverse and still avoid the stairs—if there is a ramp with just a small detour close by. Other information taken into account may be the step height of a curb (bigger wheeled robots can traverse, smaller ones cannot), the type of door (automatic, push, pull), or the material of an area (pavement, grass, sand).

Our experiment verifies the feasibility and effectiveness of the proposed global path planning method in osmAG.

This global path planning experiment is conducted in an osmAG that includes two multi-story buildings and an outdoor area. The osmAG is stored in XML file format with a size of 883.9 kb. It consists of 4908 nodes, 500 areas, and 347 passages. The actual area of the indoor area is approximately $22300 m^2$, while the outdoor area is approximately $3000 m^2$. Fig. 1 shows how we use the proposed osmAG to represent a complex and large-scale environment, and also demonstrates the global path planning results of the robot spanning two floors, across the road, leveraging semantic cues such as whether to use elevators. The path shown by the black line in the figure starts from the first floor of the building located in the bottom half of the figure, through the outdoor area to the first floor of the building in the top half of the figure, and then through the elevator to the destination located on the second floor. Our experiments are carried out on an Intel i7-6700 CPU. The distance of the planned path is 157.57m (Includes the vertical distance between floors), the time is about $3100 \mu s$ at the topological level, and the pre-compute time (calculating the metric level distance for all areas and can be applied multiple times) is about 200 ms.

VI. CONCLUSIONS

This paper proposed a novel hierarchical semantic topological map called open street map Area Graph (osmAG), based on the Area Graph and OpenStreetMap. This map represents the environment using a topometric, hierarchical semantic structure with areas as vertices and passages as edges. It can encode outdoor and indoor maps on multiple levels. Through its use of open street map format, we stay compatible with a plenitude of tools and libraries. Our osmAG C++ library is capable of loading osmAG, and visualizing and planning with it in ROS. We also will publish tools to semi-automatically generate osmAG from 2D grid maps, 3D point clouds, and CAD data.

Through utilizing osmAG, we are able to plan paths for mobile robots, taking into account their capabilities through semantic cues, and planning through multiple floors. We are able to plan very quickly by utilizing the abstract topometric and hierarchical data structure. Our experiment shows the good performance of our approach. The code and example data are available on GitHub.

For future work, we plan on providing a full ROS osmAG Navigation stack including localization, planning, and navigation with online changing of the osmAG to facilitate real-time updates of temporary or permanent map changes. We will furthermore introduce WiFi access point mapping to include those locations in the osmAG maps to fully automate our osmAG localization approach. We will then test the whole osmAG Navigation stack on real robots and provide it as open source to the robotics community.

REFERENCES

- [1] I. Kostavelis and A. Gasteratos, "Semantic mapping for mobile robotics tasks: A survey," *Robotics and Autonomous Systems*, vol. 66, pp. 86–103, 2015.
- [2] S. Thrun *et al.*, "Robotic mapping: A survey." Citeseer, 2002.
- [3] Z. He, H. Sun, J. Hou, Y. Ha, and S. Schwertfeger, "Hierarchical topometric representation of 3d robotic maps," *Autonomous Robots*, vol. 45, no. 5, pp. 755–771, 2021.
- [4] J. Hou, Y. Yuan, and S. Schwertfeger, "Area graph: Generation of topological maps using the voronoi diagram," in *19th International Conference on Advanced Robotics (ICAR)*, IEEE Press. IEEE Press, 2019.
- [5] J. Hou, Y. Yuan, Z. He, and S. Schwertfeger, "Matching maps based on the area graph," *Intelligent Service Robotics*, pp. 1–26, 2022.
- [6] S. Schwertfeger and A. Birk, "Map evaluation using matched topology graphs," *Autonomous Robots*, vol. 40, no. 5, pp. 761–787, 2016.
- [7] M. Hentschel and B. Wagner, "Autonomous robot navigation based on openstreetmap geodata," in *13th International IEEE Conference on Intelligent Transportation Systems*. IEEE, 2010, pp. 1645–1650.
- [8] L. Naik, S. Blumenthal, N. Huebel, H. Bruyninckx, and E. Prassler, "Semantic mapping extension for openstreetmap applied to indoor robot navigation," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 3839–3845.
- [9] Z. Wang and L. Niu, "A data model for using openstreetmap to integrate indoor and outdoor route planning," *Sensors*, vol. 18, no. 7, p. 2100, 2018.
- [10] Z. He, J. Hou, and S. Schwertfeger, "Furniture free mapping using 3d lidars," in *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, IEEE. IEEE, 2019.
- [11] D. Feng, Z. He, J. Hou, S. Schwertfeger, and L. Zhang, "Floorplanet: Learning topometric floorplan matching for robot localization," in *2023 International Conference on Robotics and Automation (ICRA)*, 2023.
- [12] F. Xie and S. Schwertfeger, "Robust lifelong indoor lidar localization using the area graph," in *arXiv*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.05593>
- [13] E. Michael, T. Summers, T. A. Wood, C. Manzie, and I. Shames, "Probabilistic data association for semantic slam at scale," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 4359–4364.
- [14] L. Nicholson, M. Milford, and N. Sünderhauf, "Quadricslam: Dual quadrics from object detections as landmarks in object-oriented slam," *IEEE Robotics and Automation Letters*, vol. 4, no. 1, pp. 1–8, 2019.
- [15] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, "Semanticfusion: Dense 3d semantic mapping with convolutional neural networks," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 4628–4635.
- [16] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, "Semantickitti: A dataset for semantic scene understanding of lidar sequences," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.
- [17] K. Tateno, F. Tombari, and N. Navab, "Real-time and scalable incremental segmentation on dense slam," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 4465–4472.
- [18] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: an open-source library for real-time metric-semantic localization and mapping," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 1689–1696.
- [19] B. Kuipers, "Modeling spatial knowledge," *Cognitive Science*, vol. 2, no. 2, pp. 129–153, 1978.
- [20] J.-R. Ruiz-Sarmiento, C. Galindo, and J. Gonzalez-Jimenez, "Building multiversal semantic maps for mobile robot operation," *Knowledge-Based Systems*, vol. 119, pp. 257–272, 2017.
- [21] C. Galindo, A. Saffiotti, S. Coradeschi, P. Buschka, J. Fernandez-Madrigal, and J. Gonzalez, "Multi-hierarchical semantic maps for mobile robotics," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2005, pp. 2278–2283.
- [22] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.
- [23] A. Rosinol, A. Gupta, M. Abate, J. Shi, and L. Carlone, "3d dynamic scene graphs: Actionable spatial perception with places, objects, and humans," *arXiv preprint arXiv:2002.06289*, 2020.
- [24] N. Hughes, Y. Chang, and L. Carlone, "Hydra: a real-time spatial perception system for 3d scene graph construction and optimization," 2022.
- [25] R. Bormann, F. Jordan, W. Li, J. Hampp, and M. Hägele, "Room segmentation: Survey, implementation, and analysis," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 1019–1026.
- [26] A. Kleiner, R. Baravalle, A. Kolling, P. Pilotti, and M. Munich, "A solution to room-by-room coverage for autonomous cleaning robots," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 5346–5352.
- [27] C. Liu, J. Wu, and Y. Furukawa, "Floornet: A unified framework for floorplan reconstruction from 3d scans," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 201–217.
- [28] K. M. Wurm, C. Stachniss, and W. Burgard, "Coordinated multi-robot exploration using a segmentation of the environment," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2008, pp. 1160–1165.
- [29] S. Thrun, "Learning metric-topological maps for indoor mobile robot navigation," *Artificial Intelligence*, vol. 99, no. 1, pp. 21–71, 1998.
- [30] V. Setalaphruk, A. Ueno, I. Kume, Y. Kono, and M. Kidode, "Robot navigation in corridor environments using a sketch floor map," in *Proceedings 2003 IEEE International Symposium on Computational Intelligence in Robotics and Automation. Computational Intelligence in Robotics and Automation for the New Millennium (Cat. No. 03EX694)*, vol. 2. IEEE, 2003, pp. 552–557.
- [31] S. Friedman, H. M. Pasula, and D. Fox, "Voronoi random fields: Extracting topological structure of indoor environments via place labeling," in *International Joint Conference on Artificial Intelligence*, 2007.
- [32] S. Stekovic, M. Rad, F. Fraundorfer, and V. Lepetit, "Montefloor: Extending mcts for reconstructing accurate large-scale floor plans," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 16034–16043.
- [33] T. Zheng, Z. Duan, J. Wang, G. Lu, S. Li, and Z. Yu, "Research on distance transform and neural network lidar information sampling classification-based semantic segmentation of 2d indoor room maps," *Sensors*, vol. 21, no. 4, p. 1365, 2021.
- [34] J. Hou, Y. Yuan, and S. Schwertfeger, "Topological area graph generation and its application to path planning," *arXiv preprint arXiv:1811.05113*, 2018.